

Bit Manipulation

1) Logical And (&)

a	&	b
1		0
0		1
0		0
1		1

BTW

$\hookrightarrow (n \& 1) \rightarrow$ Tell if a number is odd or even.

2) Logical or (|)

a		b
0		0
1		0
0		1
1		1

3) Logical not (~)

a	~a
1	0
0	1

4) Logical xor (^)

a	^	b
0		0
1		0
0		1
1		1

Properties of XOR

1) Identity $x \oplus 0 = x$

2) Self Inverse $x \oplus x = 0$

3) Commutative $x \oplus y = y \oplus x$

4) Associative $(x \oplus y) \oplus z = x \oplus (y \oplus z)$

5) Shift operator

a) Left Shift ($\ll$)

$$0001 \ll 1 \rightsquigarrow 0010 \rightarrow 2$$

$$0001 \ll 2 \rightsquigarrow 0100 \rightarrow 4$$

$\hookrightarrow$ multiplying with 2^i

b) Right Shift ($\gg$)

$$00001000 \gg 1 \rightsquigarrow 00000100$$

$$00001000 \gg 2 \rightsquigarrow 00000010$$

$\hookrightarrow$ dividing by 2^i

Common Bit tasks

```
bool isOdd(int n) {  
    return (n & 1) < 2;  
}
```



```

int getBit(int n, int i) {
    int mask = (1 << i); ~ 00000001 << i
    int lit = (n & mask) > 0 ? 1 : 0;
    return lit;
}

```

```

int setBit(int n, int i) {
    int mask = (1 << i);
    int ans = n | mask;
    return ans;
}

```

```

11101111
00010000
-----
11111111

```

```

void clearBit(int n, int i) {
    int mask = ~(1 << i); → 1 → 00000001
    n = n & mask;         ~1 → 11111110
}

```

```

void updateBits(int n, int i, int v) {
    int mask = ~(1 << i);
    int cleared_n = n & mask;
    n = cleared_n | (v << i);
}

```



```
int clearLastIBits(int n, int i) {
```

```
    int mask = (-1 << i);
```

```
    return n & mask;
```

-1 $\rightarrow$ 1111 1111 (Two's Complement)

```
}
```

```
int clearRangeItoJBits(int n, int i, int j) {
```

```
    int ones = ~0;
```

```
    int a = ones << (j+1);
```

```
    int b = (1 << i) - 1;
```

```
    int mask = a | b;
```

```
    int ans = n & mask;
```

```
    return ans;
```

```
}
```

$\rightarrow$

Ques $\rightarrow$ Find Unique Number.

{ 2, 2, 3, 3, 3 }